

Final Project Report
MAE 423 F25

Dini Kathriarachchi (Coding and Report Authorship)
Allison Lesser (Report Authorship and Coding)

December 18, 2025

Executive Summary

The goal of this project was to simulate the convective flow over a body using a numerical simulation. Specifically, the body experienced a low-Reynolds number, two-dimensional flow where many relevant parameters (k , c , etc.) were presumed to be constant. First, a cylindrical geometry was utilized to verify the efficacy of the coding methods, and then the arbitrary geometry was applied. The arbitrary geometries selected in this case are a set of common planforms utilized by various types of aircrafts: rectangular wings, elliptical wings and tapered wings.

The method used was the application of a two-dimensional streamfunction/vorticity finite difference model (hereinafter referred to as the Ψ - ω Method). The governing equations of this model are the Poisson's equation and the Conservation of Momentum. The former dictates the behavior of the streamfunction, the latter that of vorticity, aided by knowledge of the streamfunction in order to advance it in time. These equations were applied at each time-step in order to model the approximate behavior of the system over a predetermined time interval.

When the simulation was processed for a system with a single cylinder, vortex shedding was observed and used to determine a Strouhal number of 0.225. A research study published in the Journal of Fluid Mechanics that analyzed Strouhal numbers for flow past a cylinder at low Reynolds numbers reached a value of roughly $St = 0.2$ for Re values of ~ 200 (Hongyi & Liang 2017) [4]. The total heat transferred from the cylinder was calculated to be 23.6 kW/m^2 , from which we derived a Nusselt number of 5.91. A published value of ~ 7 was found for the Nusselt number at a Reynolds number of 200 for a similar setup with a single circular cylinder in the International Journal of Heat and Mass Transfer (Karniadakis 1988) [2]. This shows that our model is accurate to some degree with a margin of error of 12.5% for the St number and 15.6% for the Nu number.

The Strouhal numbers for the rectangular wings, elliptical wings and tapered wings were 0.220, 0.239 and 0.226 respectively, and their Nusselt numbers were 5.71, 6.00 and 5.88 respectively. Our results from the three planforms indicate that rectangular wings cause the weakest vortex shedding while elliptical wings cause the strongest vortex shedding, making rectangular wings the most efficient wing design of the three for handling air flow. Our results also show that elliptical wings allowed the most heat to be convected away from the plane while rectangular wings allowed the least, making rectangular wings the most thermally insulating wing design and the elliptical wings the most efficiently convective wing design of the three.

Methods

I. Theory

The Psi-Omega Method of numerical-simulation approach involves an eight step process within which the latter four steps repeat over the course of the desired time interval. The first step involves the definition of the mesh grid spacing and the initial boundary conditions. The grid in question has a dimension of 200 x 500 where the distance between each point was Δh . The value of Δh was determined to be 1.2×10^{-5} m (12 μ m) based on its satisfaction of the conditions:

$$Re_d = U_\infty d/\nu = 200 \ \& \ U_\infty h/\alpha < 10$$

where Re is the Reynold's number, U_∞ refers to the freestream velocity, d is the diameter of the cylinder (defined as $d=50\Delta h$), and α is the thermal diffusivity. In this simulation the Reynold's number is fixed at 200, and the thermal diffusivity and kinematic viscosity are taken to be the properties of air at room temperature with $\nu = 1.5 \times 10^{-5}$ m²/s and $\alpha = 2.2 \times 10^{-5}$ m²/s.

Once the value of h was determined, the inflow and outflow boundary conditions were set according to the following equation:

$$\Psi_{i,j} = U_\infty jh + \text{constant}$$

where the constant makes it so the streamfunction is 0 right at the center of the mesh ($j=100$). This condition is the same for both the inflow and the outflow initially and the upper and lower boundaries are just equal to the streamfunction at $i = 1$.

The second step involved determining the time step relative to the other parameters in this model. To do this, the Courant-Fredrichs-Levy condition was used to maintain numerical stability. This condition applied to this scenario stated that the time step should be less than the value over the maximum velocity which is some arbitrary multiple of the freestream velocity to ensure the simulation does not break (10 was used in this case). As such the time step was taken to be $t = 2.4 \times 10^{-7}$ s (240 ns).

The third step is to determine the initial streamfunction values assuming that the vorticity is 0. This means that the Poisson's equation simply reduces to the Laplace's equation. Then finite differences are used to approximate the differential increments where dT/dx is defined as the difference between T_{i+1} and T for instance divided by the change in x (the same is true for the y direction, just using the j parameter). Then the second rate of change is the difference in the first derivative divided by the change in x again. The resultant equation multiplied by the separation distance squared is simply the sum of the surrounding temperatures minus 2 times the temperature at that particular instance in space. If one were to add the two directions the resultant Laplace equation would be:

$$0 = \psi_{i+1,j} + \psi_{i-1,j} + \psi_{i,j+1} + \psi_{i,j-1} - 4\psi_{i,j}$$

So in calculating the streamfunction iteratively, the following equation is applied:

$$\psi_{i,j}^{k+1} = \psi_{i,j}^k + \frac{F}{4} (\psi_{i+1,j} + \psi_{i-1,j} + \psi_{i,j+1} + \psi_{i,j-1} - 4\psi_{i,j}^k)$$

where F refers to a relaxation factor which allows the streamfunction value to converge faster. If the Laplace equation is satisfied, the second term in the above equation should approximately equal zero, so this threshold can be measured by seeing how close the current streamfunction is to the previous by taking the difference. If the difference in all the streamfunction values is below the convergence criterion (a value sufficiently close to 0, taken to be 0.001 for computer processing limitations) the streamfunction values will be an accurate representation of the zero vorticity conditions.

The fourth step is applying the vorticity boundary conditions around the cylinder. Given that vorticity and velocity are defined as:

$$\omega = \frac{du}{dy} - \frac{dv}{dx} \quad \& \quad u = \frac{d\psi}{dy}, \quad v = -\frac{d\psi}{dx}$$

$$\omega = \frac{d^2\psi}{dy^2} - \frac{d^2\psi}{dx^2}$$

So the vorticity can be found using a similar methodology to that described in step 3, the only difference is that now the sum (and singular negative term) are multiplied by a constant, $-2/h^2$ reflective of the second derivative of the stream function that is being taken:

$$\omega_{i,j}^{k+1} = \frac{-2}{h^2} (\omega_{i+1,j} + \omega_{i-1,j} + \omega_{i,j+1} + \omega_{i,j-1} - 4\omega_{i,j}^k)$$

This value updates about the boundary of the cylinder, the bulk values of vorticity propagate via the diffusion of momentum as well as the flow field as seen in the following step.

The fifth step is where the conservation of momentum is applied to find the bulk values of vorticity. This equation involves the diffusive term as well as the upwind difference which results from the velocity. The equation is the following:

$$\omega_{i,j}^{k+1} = \omega_{i,j}^k + \Delta t \left(\frac{-(\Delta u \omega)^n}{h} + \frac{-(\Delta v \omega)^n}{h} + v \left(\frac{d^2 \omega}{dx^2} + \frac{d^2 \omega}{dy^2} \right) \right)$$

Where the definition of the upwind difference terms depends on whether the velocity is in the positive or negative direction. All of the terms utilize the finite difference approximation method such that the derivative or Δ terms are the difference between the relevant surrounding parameters. After this the streamfunction value is updated in accordance with the finite difference approximation solution of the Poisson equation, where the resultant equation is:

$$\psi_{i,j}^{k+1} = \psi_{i,j}^k + \frac{F}{4} (\psi_{i+1,j} + \psi_{i-1,j} + \psi_{i,j+1} + \psi_{i,j-1} + 4h^2 \omega_{i,j}^{k+1} - 4\psi_{i,j}^k)$$

The latter term converging is again indicative of the satisfaction of the relevant equation in this case, so the above equation is performed iteratively until the term meets the convergence criterion. That value was also taken to be 0.001 in this case.

The sixth step is to calculate the outflow boundary conditions. At the outflow boundary, it is assumed that the velocity is constant and the vorticity is convected out. Therefore the resultant equations are used:

$$\psi_{i,j}^{k+1} = 2\psi_{i-1,j}^k - \psi_{i-2,j}^k$$

$$\omega_{i,j}^{k+1} = \omega_{i-1,j}^k$$

Where the updated streamfunction is computed based on the value in the bulk of the flow and the vorticity is simply the preceding term.

The seventh step is to update the temperature field based on the conservation of energy. Two dimensional heat transfer obeys the following equation under constant conditions:

$$\frac{\partial T}{\partial t} + u \frac{\partial T}{\partial x} + v \frac{\partial T}{\partial y} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right)$$

So based on this equation and the finite difference approximation, an analogous equation to that which was used for vorticity can be applied here to find the updated temperature over the entire flow field. The equation is the following:

$$T_{i,j}^{n+1} = T_{i,j}^n + \Delta t \left(-u_{i,j} \frac{(\Delta T)_{i,j}}{h} - v_{i,j} \frac{(\Delta T)_{i,j}}{h} + \alpha \left(\frac{d^2 T}{dx^2} + \frac{d^2 T}{dy^2} \right) \right)$$

Depending on the sign of the velocity, which temperature values are taken to be the relevant adjacent ones changes. The temperature values are updated over the entire grid based on this equation, where the changes result from both the diffusion and upwind difference terms again.

Finally steps four through seven are repeated for the desired number of time steps to understand how the streamfunction, vorticity, and temperature evolve over time.

II. Code and Simulation

MATLAB was used to construct the simulation, gather numerical data and generate video files to visualize the progression of the streamfunction, vorticity and temperature within the system. The code is split into 4 sections as follows: initialization, timestep iteration, video generation and grid iteration functions.

The initialization section contains code to construct data structures required to store the large amount of numerical data that is being generated and computed. In this section, all physical constants (such as freestream velocity u_{inf} , thermal diffusivity α , kinetic viscosity ν , Reynold's number Re , etc) required for system calculations are set, as well as technical parameters with regards to the simulation (such as time intervals, convergence tolerance ϵ , grid dimension $n \times m$, etc). Initial physical conditions (such as initial bulk temperature, streamfunction values at all 4 walls, etc) are also incorporated into the model here. Lastly, the rigid objects within the system around which the fluid flows are encoded using a boolean (true/false) grid that indicates whether the corresponding coordinate lies within the object or not.

Four simulations were executed to investigate fluid and heat flow around four different 2D structures. The first was a simple circular cylinder with a radius of $25 \cdot \Delta h$, an initial test case to verify that the model functioned as desired and produced reliable output. Fluid flow and heat flow constants St and Nu calculated from this initial model were then compared to existing, published values for these constants to validate the model. We were interested in investigating how the wing-shape of aircrafts affected (i) their ability to travel through air while minimizing induced turbulence and (ii) their rate of convective heat loss. To explore this, three wing shape designs (Fig 1) were tested: rectangular wings (standard rectangle), ellipsoid wings (standard ellipse) and tapered wings (rectangular wings with front edge tapered). The body of the airplane

was modelled as a fixed horizontal ellipsoid at the same coordinates for all three models to ensure the only change between the models was wing shape.

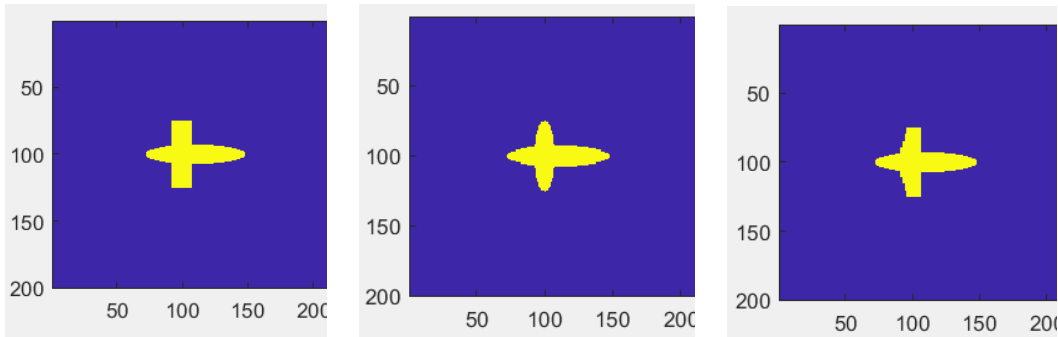


Figure 1: (left) Rectangular wings, (center) ellipsoid wings & (right) tapered wings

The timestep iteration loop enables the system to progress temporally through the calculations discussed in the prior section. Data from the previous time step is used to determine how the physical parameters will evolve in the current time step. To improve clarity and flow of logic, the loop contains code that calls the three grid iteration functions which iterate through the streamfunction, vorticity and temperature grids, and recalculate and update their values using the simplified theoretical relationships. The last section of the code serves to generate a media file that allows us to visualize how the three properties evolve spatially (within the grid) and temporally. This section consists of mostly high-level code commands that execute pre-defined subroutines to allow for the grids to be displayed, colored coded, structured into picture frames and incorporated into a MPEG-4 video file spanning just over 30 seconds. Debugging the code was unfortunately a time-consuming and cumbersome process and involved having to revisit the theoretical relationships between variables and comb through the 250 line code carefully several times to identify causes of error and inaccuracy.

III. Model Validation Calculations

As mentioned above, to verify the reliability of the fluid flow and heat flow models of this simulation, we chose to calculate the Strouhal number and the Nusselt number. The former quantifies oscillating fluid flow mechanisms (e.g. vortex shedding) and informs us about the properties of fluid flow within the system, while the latter quantifies convective heat transfer relative to conductive heat transfer and informs us about the heat flow properties of the system. The Strouhal number, St , can be calculated as follows:

$$St = f * L / U$$

where f is the frequency of vortex shedding, L is the length scale and U is the fluid velocity. f can be determined as $1/T$ where T is the time (in seconds) from when a vortex begins to form to when it detaches from the rest of the heat flow. The length scale L is simply the diameter for the cylinder and the wingspan length for the three planforms. The Nusselt number, Nu , is a bit more complicated to determine.

$$Nu = hL/k$$

where L is the length scale and k is the thermal conductivity of air. However, h is the coefficient of convective heat transfer which must be calculated using the following equation:

$$h = \Delta Q / (T_{obj} - T_{inf})$$

where T_{body} is the temperature of the object and T_{inf} is the temperature of the flowing air. ΔQ will have to be computed using values from the temperature grid and two velocity grids as follows:

$$\Delta Q = \rho_{air} * c_p * ([uT]_{out} - [uT]_{in})$$

where $[uT]_{out}$ and $[uT]_{in}$ are the averaged velocity-temperature products at the left and right boundaries of the system. Using these dimensionless St and Nu values, we can make a judgement regarding the accuracy of the system output by comparing them to known, published values.

Results

I. Qualitative

The following results primarily discuss the evolution of the quantities that were visually represented in the flow field as well as how these results compared to the expected values (primarily that of the Strouhal number).

First, in regards to the streamfunction, the initial representation should represent the solution to Laplace's equation if it reflects what was expected. This means that the boundary conditions should propagate until the solution is met. What this is expected to look like is a series of lines tangent to the cylinder where the values below the centerline were negative and above the centerline were positive:

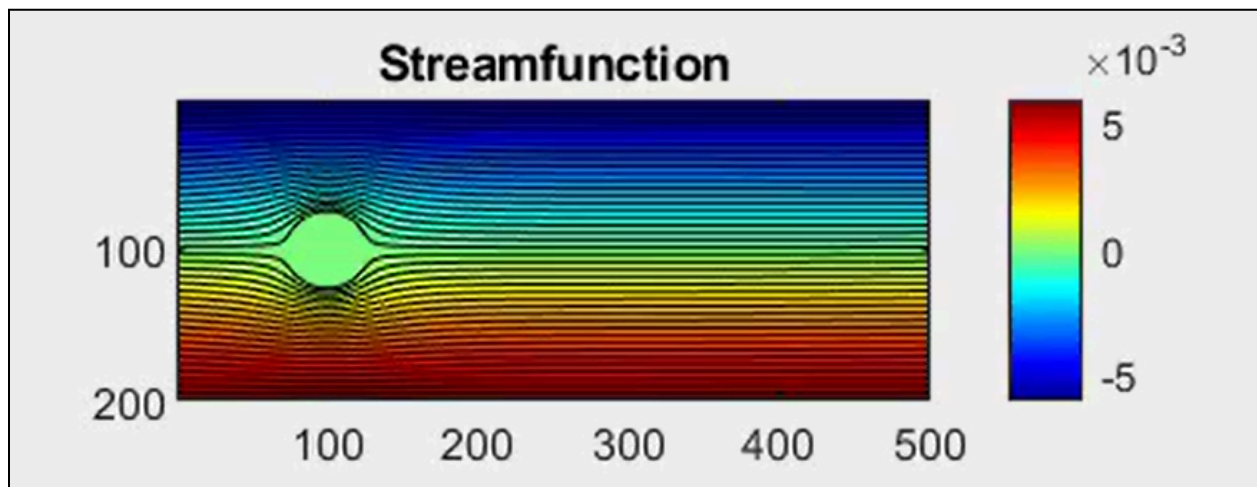


Figure 2: Initial Streamfunction Graph

The figure shown above depicts the exact behavior of the stream function which was expected.

As time progresses and the influence of vorticity is introduced to the system, one can see the streamfunction begin to separate immediately behind the cylinder. This behavior is also expected as it directly precedes the formation of the vortex street. This separation manifests in the stream function as the initial formation of ellipse-like regions directly after the cylinder:

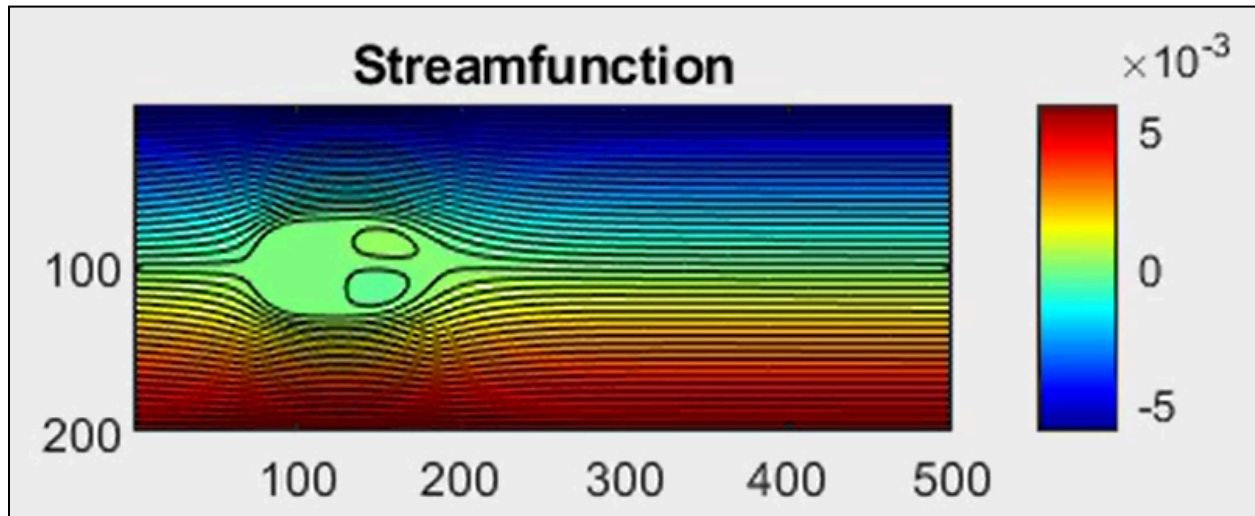


Figure 3: Streamfunction Shortly after the Introduction of Vorticity

While the two regions appear to be somewhat symmetric at the start, one can see the genesis of the lack of symmetry which results in the formation of the vortex street. This appears first as the expansion of the closed regions in both the x and y directions and then the larger of the two formations transitions into a wave. This begins an oscillatory pattern in the wake which persists for the rest of the simulation.

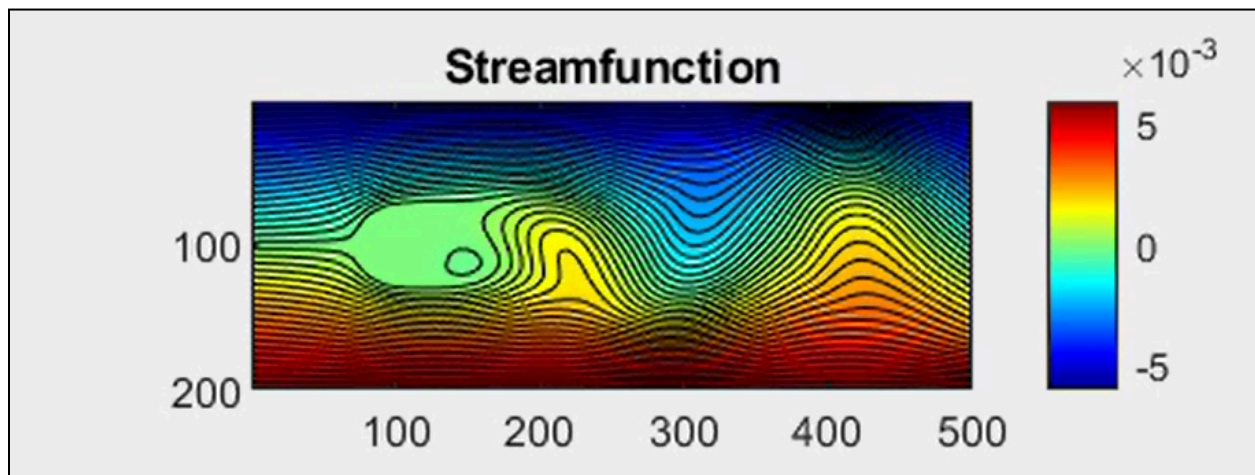


Figure 4: Continued Evolution of the Streamfunction

The evolution of the stream function appears to be consistent with the expectations set by the solutions to both the Laplace Equation as well as the Poisson equation throughout the simulation. Thus it is concluded that this part of the solution is reasonable.

The vorticity begins with the non-zero boundary condition as the only grid points not initialized to zero. The vorticity then propagates through the flow via the influence of both the diffusivity and the velocity which causes the upwind difference. Once the boundary conditions are established, the momentum equation is then applied, first small regions form at the back region of the cylinder. Then, as more time passes, a vortex street matching that of the streamfunction forms.

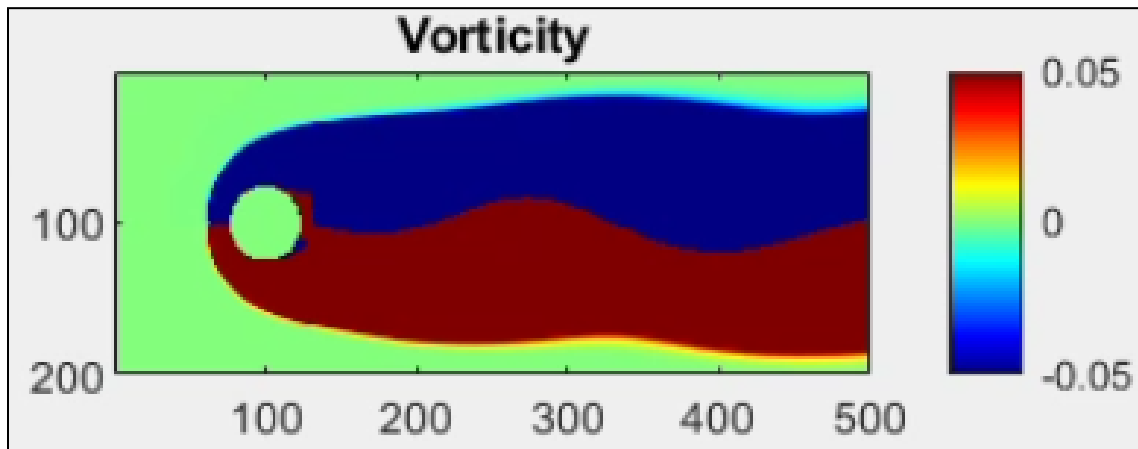


Figure 5: Evolving vorticity

The image above conforms to the cyclical nature that is to be expected corresponding to the wake of the cylinder. As was seen in the streamfunction before. The vorticity begins simply by diffusing off of the surface of the cylinder originating from the non-zero boundary conditions, the vorticity then propagates downstream while continuing to diffuse into the bulk. The vorticity then continues to fluctuate before it is eventually convected out of the downstream boundary. This model is presumed to be correct initially given the nature of the signs of the vorticity throughout the model. It is consistent with the mathematical model that the value of vorticity should be approximately positive above the centerline and negative below it. While the “vortex street” is not eminently observed, the influence of vorticity leads to that response in the temperature which is explored in the subsequent section.

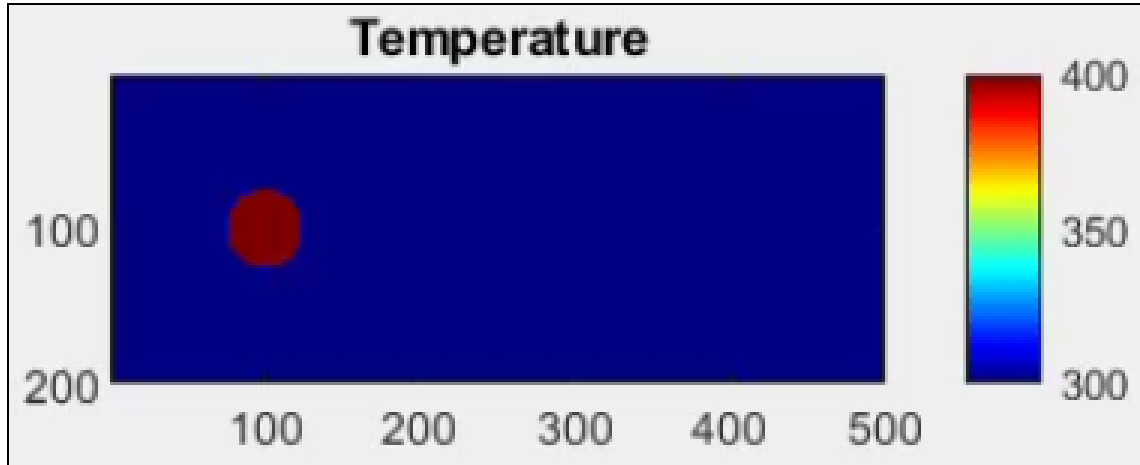


Figure 6: Initial temperature distribution

As for the development of the temperature profile, it begins as expected at the instance of time being zero: the higher temperature cylinder is the only area that is not the temperature of the bulk. It remains at 400K in the approximately room temperature environment of 300K. Then as the time steps begin to advance, one can see the heat begin to diffuse first into the air immediately surrounding it, this manifesting in a reddish-orange ring. However, in the immediate wake, the temperature begins to move downwind due to the upwind differencing component. The temperature continues to propagate to the left and due to the combined contributions of both components of the temperature equation, the characteristic “vortex street” becomes evident.

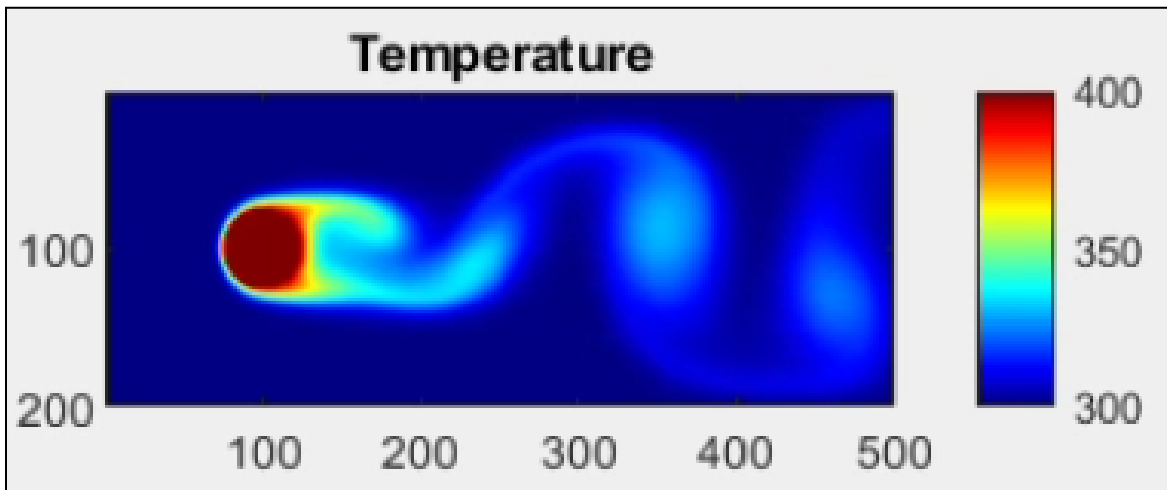


Figure 7: Evolving flow of heat

As the heat is successfully convected away from the cylinder, the average amount of time between each vortex becomes evident. It is this element of the simulation that informed the mathematical elements of the subsequent section, namely the calculation of the Strouhal and Nusselt numbers.

II. Quantitative

The Strouhal numbers were determined using the formulae highlighted earlier. The video files were analyzed and the fluid and heat flow were allowed time to develop (~5000 timesteps) and exhibit the consistent oscillatory pattern of a vortex street. The vortices were easiest to distinguish in the temperature simulation and this grid was observed in order to measure the period of a single vortex shed. For all four simulations, the time period of a vortex shed was between 2000 to 2500 timesteps equivalent to roughly 0.5 to 0.6 ms. The length scale in all four simulations was determined to be $50 \cdot \Delta h$ which is equivalent to 0.6 mm. The equation $St = f \cdot L / U$ was used to deduce the following St values for the four simulations:

Simulated Object	Calculated Strouhal Number
Single Circular Cylinder (Diameter $50 \cdot \Delta h$)	0.225
Airplane with Rectangular Wings (Wingspan $50 \cdot \Delta h$)	0.220
Airplane with Ellipsoid Wings (Wingspan $50 \cdot \Delta h$)	0.239
Airplane with Tapered Wings (Wingspan $50 \cdot \Delta h$)	0.226

The Nusselt number was deduced using a code (refer to Appendix) which incorporated the equations introduced in the earlier section. It considered the column of grid points at the inflow boundary and the column of grid points at $x = 300$ (3.6 mm). The outflow boundary was at $x = 500$ could have alternatively been used for this calculation to yield a similar result. However, the simulation could only be run for 10,000 timesteps (2.4 ms) and the first vortex reached this outflow boundary at ~4500 (1.08 ms) in the temperature video, meaning the outflow boundary would remain fairly unchanged for almost half of the run time, resulting in a much smaller average rate of heat loss and, by extension, a smaller Nusselt number. For each horizontal row of the grid, the difference between the velocity-temperature products at the inflow boundary and outflow (choosing $x = 300$) boundary was computed and used to determine the rate of heat loss from each grid column. The average rate of heat loss from the system was determined by computing the mean heat loss from a grid point in the direction parallel to the inflow U_{inf} . Lastly, the Nusselt number was computed using the combined formula shown below:

$$Nu = [\Delta Q_{avg} \cdot L] / [k \cdot (T_{obj} - T_{inf})]$$

where $L = 50 \cdot \Delta h$, $T_{obj} = 400$ K and $T_{inf} = 300$ K, and the final values obtained for each simulation were as follows:

Simulated Object	Calculated Strouhal Number
Single Circular Cylinder (Diameter $50 \cdot \Delta h$)	5.91
Airplane with Rectangular Wings (Wingspan $50 \cdot \Delta h$)	5.71
Airplane with Ellipsoid Wings (Wingspan $50 \cdot \Delta h$)	6.00
Airplane with Tapered Wings (Wingspan $50 \cdot \Delta h$)	5.88

Discussion and Conclusion

While a reasonable degree of confidence can be had in the extent of the results that can be empirically verified through more definitive sources, it should be noted there were significant challenges in the process of acquiring them. The largest of which seemed to be the timing necessary to run the simulations in question.

I. Challenges

This is not a shocking revelation in retrospect, however all of the simulations took an extremely long time to receive results from. Initially a simulation of approximately 2000 timesteps took around 3 hours to run. Then on a different device with slightly more processing power, the same simulation took 2 hours to iterate over 6000 timesteps. While this was significantly faster, it was still woefully slow when compared to word of other coded simulations running simultaneously for other iterations of this very project (with one report claiming to be able to run 75,000 timesteps in 1 hour). Therefore, it was concluded that this was a unique challenge to the code, if not a bug, then something hindering the ability of the participants to collect a sufficient amount of data. As it turns out, the error lay not within the code but the assigning of a specific value: the convergence criterion.

While most models had opted to utilize the conventional value of 0.01 (or perhaps even 0.1), this specific code had been utilizing a value of 0.001. This may have yielded marginally more accurate results, but it significantly slowed down the running of the program, given that when the streamfunction needed to converge, it would have required additional calls to the psi grid iteration function. Likewise, if one was to multiply that increase (say 3 for the sake of argument) by the total number of values in the grid (200×500), this would involve an additional 300,000 computational steps—600,000 if one was to count both Laplace's equation and the Poisson Equation. Therefore to reduce the amount of iterations needed and thus the time needed to do them, eventually the value of 0.01 was adopted. This choice significantly reduced the runtime of the program without compromising too much on accuracy. It also allowed for more timesteps to be simulated per each scenario.

Another challenge we encountered that took time to resolve was the occurrence of symmetric vortices in the simulation (Fig 8) that continued to propagate through the system without destabilizing into an oscillatory vortex street.

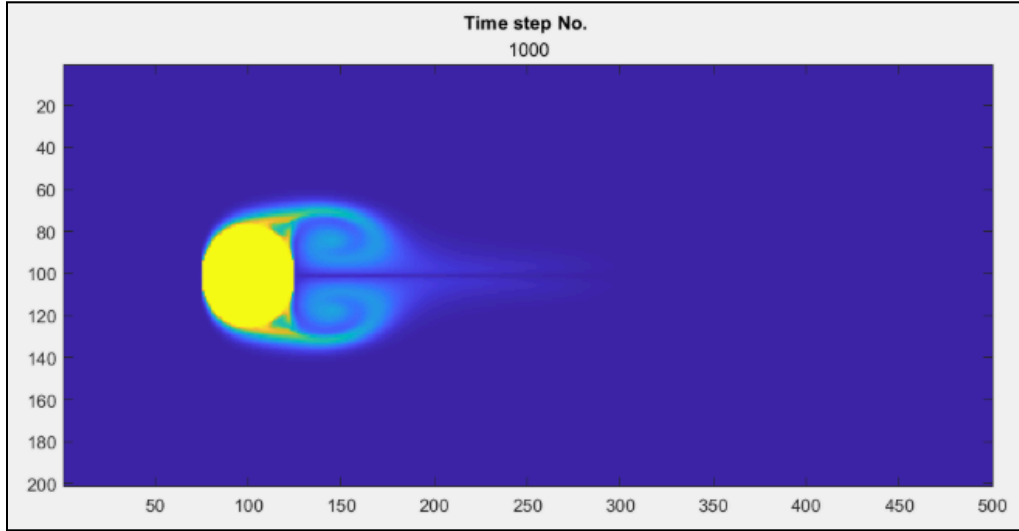


Figure 8: Symmetric vortices error

We later understood that this was because of the way we were updating vorticity within the simulation, as we were only using values from the earlier timestep instead of the values that were simultaneously being updated as is seen in Gauss-Siedel iterations.

II. Model Accuracy

The first of the four simulations, which involves the circular cylinder, served to verify the validity and accuracy of the numerical output produced by the model. To test whether the model was able to correctly represent fluid and heat flow theories, we computed the Strouhal number to ensure fluid motion/dynamics correctness and the Nusselt number to confirm thermal transfer correctness. The St number computed for the cylinder by our model was 0.225. Hongyi & Liang (2017) [4] published a paper in the Journal of Fluid Mechanics that analyzed Strouhal numbers for air flow past a circular cylinder at low Reynolds numbers and arrived at a value of roughly $St = 0.2$ for Re values of ~ 200 . If we assume this value is a suitable representation of what the actual Strouhal number should be in such a system, then our St value, and by extension our model, would be acceptable within a margin of error of 12.5%. Karniadakis (1988) [2] also numerically simulated forced convective heat transfer around a circular cylinder and deduced that the Nusselt number of the system at a low Reynold's value of ~ 200 is around 7. Our mathematical model yielded a Nusselt number of 5.91, so we are to assume that $Nu = 7$ is a precise representation of the heat flow within this system, our result is acceptable within a margin of error of 15.6%. These two margins of error, while not entirely negligible, are small enough that we can conclude our results to be a close approximation to the actual systems they simulate. A possible source of numerical error may be that we reduced differential relations

between physical quantities such as streamfunction and vorticity into simple algebraic formulae which would have underrepresented the continuous nature of real world physical systems. However, this is not something we would address in a numerical simulation and the digital processors that perform these computations require finite, discrete quantities unlike those that manifest in the real world.

III. Model Applicability and Future Experimentation

The latter elements of this simulation were selected from an inherent interest in aerospace applications as well as curiosity for the reasoning behind certain planform design choices. Different aircrafts have different wingshapes for a variety of reasons. For instance, tapered wings tend to be a lot more common in high speed aircrafts while lighter aircrafts may utilize rectangular wings. While such design decisions are often informed by other elements—the tapered wings corresponding to lower drag force—rather than temperature, it is still important to take it into consideration.

For example, in the instance of an aircraft wing significantly exceeding the temperature of the ambient environment, the rate of heat flux off of it becomes important. This is true because above certain temperatures a wing might experience problems relating to the thermal expansion of the materials involved which could lead to mechanical failure. Since the airfoils (or at least the cross-sectional area) is relatively similar between these applications, one can look at the planform to see if there are any significant differences. In the case of this project, high temperatures or high velocity flows are not specifically studied, however there is no immediate indication that this sort of model would not be useful in such an instance. The model would likely have to be updated to account for elements such as radiation (since the temperature difference is large) however the method of simulation and programming in general should be largely applicable. Thus, future individuals could create these simulations to ensure that such aerospace applications don't fail at exceedingly high temperatures.

References

- [1] Glazer, Neil. “Explore 10 Unique Types of Aircraft Wings for Better Flight.” *PilotMall.Com*, 2 Nov. 2023, www.pilotmall.com/blogs/news/aircraft-wing-design-10-types-of-aircraft-wings-complete-guide?srltid=AfmBOoojNLqfTSPbSJ2i6KsnGTXDNsTUIsUjwvXEzEOc0gobmfO_kc-.
- [2] George E M Karniadakis “Numerical simulation of forced convection heat transfer from a cylinder in crossflow.” *International Journal of Heat and Mass Transfer*, Volume 31, Issue 1, 1988, Pages 107-118, ISSN 0017-9310, [https://doi.org/10.1016/0017-9310\(88\)90227-X](https://doi.org/10.1016/0017-9310(88)90227-X).
- [3] Holman, J.P. *Heat Transfer*. Tenth ed., McGraw-Hill Education, 2009.
- [4] Jiang, Hongyi, and Liang Cheng. “Strouhal–Reynolds Number Relationship for Flow Past a Circular Cylinder.” *Journal of Fluid Mechanics* 832 (2017): 170–188. Web.
- [5] Nosenchuck, Dan. “Numerical Simulation of 2-D Unsteady Convective Heat Transfer.” *Princeton University Canvas - MAE 423*, 2025, princeton.instructure.com/courses/19036/files/4159199?module_item_id=799522.

Appendix

Video Files:

1) Single cylinder:

 CylinderVideoMAE423.mp4

2) Plane with rectangular wings:

 RectangleVideoMAE423.mp4

3) Plane with elliptical wings:

 EllipsoidVideoMAE423.mp4

4) Plane with tapered wings:

 TaperedVideoMAE423.mp4

Code:

1) Single cylinder:

```
% MAE 423 Final Project
%
% By Dini Kathriarachchi & Allison Lesser
% set constants for problem
n = 500; % vertical length
m = 200; % horizontal length
```

```

threshold_error = 0.05;
F_overrelax = 1.5;
time_intervals = 10000;
numFrame = 0;
frame_store_rate = 10; % how many frames are stored
% constants for air
U_inf = 5; % m/s
T_inf = 300; % K
therm_diffusivity = 2.2 * 10^-5; % m^2/s
kin_viscosity = 1.5 * 10^-5;
Re = 200; % no units
delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf);
% create cell to store frames
psicell = cell(ceil(time_intervals/frame_store_rate), 1);
omegacell = cell(ceil(time_intervals/frame_store_rate), 1);
tempcell = cell(ceil(time_intervals/frame_store_rate), 1);
% create grids of desired geometry (n x m)
psiGrid = zeros(n, m);
omegaGrid = zeros(n, m);
uGrid = zeros(n, m);
vGrid = zeros(n, m);
tempGrid = T_inf*ones(n, m);
objGrid = false(n, m);
%
% create cylinder
radius = 25;
cyl_cen_x = 100;
cyl_cen_y = ceil(m/2);
%
for i = 1 : n
    for j = 1 : m
        if (sqrt((i - cyl_cen_x)^2 + (j - cyl_cen_y)^2) < radius)
            objGrid(i, j) = true;
        end
    end
end
% set boundary conditions
for j = 1 : m % left and right boundaries
    psiGrid(1, j) = (j - ceil(m/2)) * delta_h * U_inf;
    psiGrid(n, j) = (j - ceil(m/2)) * delta_h * U_inf;
end
% top and bottom boundaries
psiGrid(:, 1) = (1 - ceil(m/2)) * delta_h * U_inf;
psiGrid(:, m) = (m - ceil(m/2)) * delta_h * U_inf;
% temp boundaries
for j = 2 : m - 1
    for i = 2 : n - 1
        if objGrid(i, j)

```



```

        tempGrid(i, j) = 400;
    end
end
end
%% model changes begin
for time = 1 : time_intervals
    % update u and v values for omega calculation
    for j = 2 : m - 1
        for i = 2 : n - 1
            uGrid(i, j) = (psiGrid(i, j+1) - psiGrid(i, j-1))/(2*delta_h);
            vGrid(i, j) = (psiGrid(i-1, j) - psiGrid(i+1, j))/(2*delta_h);
        end
    end
    % iterate through omega grid until errors are below threshold
    omegaGrid = iterate_omegagrid(psiGrid, objGrid, omegaGrid, uGrid, vGrid,
delta_h, delta_t, kin_viscosity, n, m);
    % iterate through omega grid until errors are below threshold
    tempGrid = iterate_tempgrid(objGrid, tempGrid, uGrid, vGrid, delta_h,
delta_t, therm_diffusivity, n, m);
    % iterate through psi grid until errors are below threshold
    max_error = Inf;
    while max_error > threshold_error
        [psiGrid, max_error] = iterate_psigrid(psiGrid, objGrid, omegaGrid,
delta_h, n, m, F_overrelax);
    end
    % store every 1 in few frames to avoid overloading :)
    if mod(time, frame_store_rate) == 1
        numFrame = numFrame + 1;
        psicell{numFrame} = psiGrid;
        omegacell{numFrame} = omegaGrid;
        tempcell{numFrame} = tempGrid;
        disp("Created frame " + string(numFrame) + " at time " +
string(datetime('now')));
    end
end
%% create video
videoFinal = VideoWriter('CylinderVideoMAE423.mp4', 'MPEG-4');
open(videoFinal);
% loop through frames and store them
for f = 1 : numFrame
    psi = psicell{f}';
    omega = flipud(omegacell{f}');
    temp = tempcell{f}';
    subplot(3, 1, 1)
    cla;
    imagesc(psi);
    hold on
    contour(psi, 40, 'k');
    hold off

```

```

colormap(jet);
axis equal tight;
colorbar;
title('Streamfunction');
drawnow;
subplot(3, 1, 2)
cla;
imagesc(omega)
colormap(jet);
axis equal tight;
ax = gca; % Get current axes handle
ax.CLim = [-0.05, 0.05]; % Set color limits
colorbar;
title('Vorticity');
drawnow;
subplot(3, 1, 3)
cla;
imagesc(temp)
colormap(jet);
axis equal tight;
colorbar;
title('Temperature');
drawnow;
sgtitle("Time step No. " + string(f*frame_store_rate - 4));
frame = getframe(gcf); % get current frame
writeVideo(videoFinal, frame); % add frame to video
end
close(videoFinal);
%% iterate through omega grid
function newGrid = iterate_omegagrid(psiGrid, objGrid, oldOmegaGrid, u, v, h,
t, nu, n, m)
    modOmegaGrid = oldOmegaGrid;
    for j = 2 : m - 1
        for i = 2 : n - 1
            if objGrid(i, j)
                modOmegaGrid(i, j) = -2*(psiGrid(i+1, j) + psiGrid(i-1, j) +
psiGrid(i, j+1) + psiGrid(i, j-1) - 4*psiGrid(i, j))/(h^2);
            end
        end
    end
    for j = 2 : m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)
                if u(i, j) < 0
                    delta_u_omega = u(i+1, j)*modOmegaGrid(i+1, j) - u(i,
j)*modOmegaGrid(i, j);
                else
                    delta_u_omega = u(i, j)*modOmegaGrid(i, j) - u(i-1,
j)*modOmegaGrid(i-1, j);

```

```

        end
        if v(i, j) < 0
            delta_v_omega = v(i, j+1)*modOmegaGrid(i, j+1) - v(i,
j)*modOmegaGrid(i, j);
        else
            delta_v_omega = v(i, j)*modOmegaGrid(i, j) - v(i,
j-1)*modOmegaGrid(i, j-1);
        end
        laplacian_omega = (modOmegaGrid(i+1, j) + modOmegaGrid(i-1, j) +
modOmegaGrid(i, j-1) + modOmegaGrid(i, j+1) - 4*modOmegaGrid(i, j))/(h^2);
        modOmegaGrid(i, j) = modOmegaGrid(i, j) +
t*(-1*(delta_u_omega)/h - (delta_v_omega)/h + nu*laplacian_omega);
    end
end
end
modOmegaGrid(n, :) = modOmegaGrid(n-1, :);
newGrid = modOmegaGrid;
end
%% iterate through temp grid
function newGrid = iterate_tempgrid(objGrid, oldTempGrid, u, v, h, t, alpha, n,
m)
    modTempGrid = oldTempGrid;
    for j = 2 : m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)
                if u(i, j) < 0
                    delta_u_temp = u(i, j)*modTempGrid(i+1, j) - u(i,
j)*modTempGrid(i, j);
                else
                    delta_u_temp = u(i, j)*modTempGrid(i, j) - u(i,
j)*modTempGrid(i-1, j);
                end
                if v(i, j) < 0
                    delta_v_temp = v(i, j)*modTempGrid(i, j+1) - v(i,
j)*modTempGrid(i, j);
                else
                    delta_v_temp = v(i, j)*modTempGrid(i, j) - v(i,
j)*modTempGrid(i, j-1);
                end
                laplacian_temp = (modTempGrid(i+1, j) + modTempGrid(i-1, j) +
modTempGrid(i, j-1) + modTempGrid(i, j+1) - 4*modTempGrid(i, j))/(h^2);
                modTempGrid(i, j) = modTempGrid(i, j) + t*(-1*(delta_u_temp)/h -
(delta_v_temp)/h + alpha*laplacian_temp);
            end
        end
    end
    modTempGrid(n, :) = modTempGrid(n-1, :);
    newGrid = modTempGrid;
end

```

```

%% iterate through psi grid using over-relaxed finite-difference (Gauss-Seidel)
and also error
function [newGrid, max_error] = iterate_psigrid(oldPsiGrid, objGrid, omegaGrid,
h, n, m, F)
    modPsiGrid = oldPsiGrid;
    errorGrid = zeros(n, m);
    for j = 2 : m - 1
        for i = 2 : n - 1
            if objGrid(i, j)
                continue;
            end
            % determine psi
            modPsiGrid(i, j) = modPsiGrid(i, j) + F*0.25*(modPsiGrid(i-1, j) +
modPsiGrid(i+1, j) + modPsiGrid(i, j-1) + modPsiGrid(i, j+1) +
4*h*h*omegaGrid(i, j) - 4*modPsiGrid(i, j));
            % determine error
            errorGrid(i, j) = abs(modPsiGrid(i, j) - oldPsiGrid(i, j)) /
oldPsiGrid(i, j);
        end
        modPsiGrid(n, j) = 2*modPsiGrid(n-1, j) - modPsiGrid(n-2, j);
    end
    newGrid = modPsiGrid;
    max_error = max(errorGrid, [], 'all');
end

```

2) Plane with rectangular wings:

```

% MAE 423 Final Project
%
% By Dini Kathriarachchi & Allison Lesser
% set constants for problem
n = 500; % vertical length
m = 200; % horizontal length
threshold_error = 0.05;
F_overrelax = 1.5;
time_intervals = 10000;
numFrame = 0;
frame_store_rate = 10; % how many frames are stored
% constants for air
U_inf = 5; % m/s
T_inf = 300; % K
therm_diffusivity = 2.2 * 10^-5; % m^2/s
kin_viscosity = 1.5 * 10^-5;
Re = 200; % no units
delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf);
% create cell to store frames
psicell = cell(ceil(time_intervals/frame_store_rate), 1);
omegacell = cell(ceil(time_intervals/frame_store_rate), 1);

```

```

tempcell = cell(ceil(time_intervals/frame_store_rate), 1);
% create grids of desired geometry (n x m)
psiGrid = zeros(n, m);
omegaGrid = zeros(n, m);
uGrid = zeros(n, m);
vGrid = zeros(n, m);
tempGrid = T_inf*ones(n, m);
objGrid = false(n, m);
%
% rectangle wings
wlength = 50;
wwidth = 15;
coord_x = 100;
for i = coord_x - ceil(wwidth/2) : coord_x + floor(wwidth/2)
    for j = ceil(m/2) - ceil(wlength/2) : ceil(m/2) + floor(wlength/2)
        objGrid(i, j) = true;
    end
end
% plane body
plength = 15;
pwidth = 75;
coord_x = 110;
for i = 1 : n
    for j = 1 : m
        if ((j - ceil(m/2))^2/(plength/2)^2 + (i - coord_x)^2/(pwidth/2)^2 <= 1)
            objGrid(i, j) = true;
        end
    end
end
% set boundary conditions
for j = 1 : m % left and right boundaries
    psiGrid(1, j) = (j - ceil(m/2)) * delta_h * U_inf;
    psiGrid(n, j) = (j - ceil(m/2)) * delta_h * U_inf;
end
% top and bottom boundaries
psiGrid(:, 1) = (1 - ceil(m/2)) * delta_h * U_inf;
psiGrid(:, m) = (m - ceil(m/2)) * delta_h * U_inf;
% temp boundaries
for j = 2 : m - 1
    for i = 2 : n - 1
        if objGrid(i, j)
            tempGrid(i, j) = 400;
        end
    end
end
end
%% model changes begin
for time = 1 : time_intervals
    % update u and v values for omega calculation
    for j = 2 : m - 1

```

```

        for i = 2 : n - 1
            uGrid(i, j) = (psiGrid(i, j+1) - psiGrid(i, j-1))/(2*delta_h);
            vGrid(i, j) = (psiGrid(i-1, j) - psiGrid(i+1, j))/(2*delta_h);
        end
    end
    % iterate through omega grid until errors are below threshold
    omegaGrid = iterate_omegagrid(psiGrid, objGrid, omegaGrid, uGrid, vGrid,
    delta_h, delta_t, kin_viscosity, n, m);
    % iterate through omega grid until errors are below threshold
    tempGrid = iterate_tempgrid(objGrid, tempGrid, uGrid, vGrid, delta_h,
    delta_t, therm_diffusivity, n, m);
    % iterate through psi grid until errors are below threshold
    max_error = Inf;
    while max_error > threshold_error
        [psiGrid, max_error] = iterate_psigrid(psiGrid, objGrid, omegaGrid,
    delta_h, n, m, F_overrelax);
    end
    % store every 1 in few frames to avoid overloading :)
    if mod(time, frame_store_rate) == 1
        numFrame = numFrame + 1;
        psicell{numFrame} = psiGrid;
        omegacell{numFrame} = omegaGrid;
        tempcell{numFrame} = tempGrid;
        disp("Created frame " + string(numFrame) + " at time " +
    string(datetime('now')));
    end
end
%% create video
videoFinal = VideoWriter('RectangleVideoMAE423.mp4', 'MPEG-4');
open(videoFinal);
% loop through frames and store them
for f = 1 : numFrame
    psi = psicell{f}';
    omega = flipud(omegacell{f}');
    temp = tempcell{f}';
    subplot(3, 1, 1)
    cla;
    imagesc(psi);
    hold on
    contour(psi, 40, 'k');
    hold off
    colormap(jet);
    axis equal tight;
    colorbar;
    title('Streamfunction');
    drawnow;
    subplot(3, 1, 2)
    cla;
    imagesc(omega)

```

```

colormap(jet);
axis equal tight;
ax = gca; % Get current axes handle
ax.CLim = [-0.05, 0.05]; % Set color limits
colorbar;
title('Vorticity');
drawnow;
subplot(3, 1, 3)
cla;
images(temp)
colormap(jet);
axis equal tight;
colorbar;
title('Temperature');
drawnow;
sgtitle("Time step No. " + string(f*frame_store_rate - 4));
frame = getframe(gcf); % get current frame
writeVideo(videoFinal, frame); % add frame to video
end
close(videoFinal);
%% iterate through omega grid
function newGrid = iterate_omegagrid(psiGrid, objGrid, oldOmegaGrid, u, v, h,
t, nu, n, m)
    modOmegaGrid = oldOmegaGrid;
    for j = 2 : m - 1
        for i = 2 : n - 1
            if objGrid(i, j)
                modOmegaGrid(i, j) = -2*(psiGrid(i+1, j) + psiGrid(i-1, j) +
psiGrid(i, j+1) + psiGrid(i, j-1) - 4*psiGrid(i, j))/(h^2);
            end
        end
    end
    for j = 2 : m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)
                if u(i, j) < 0
                    delta_u_omega = u(i+1, j)*modOmegaGrid(i+1, j) - u(i,
j)*modOmegaGrid(i, j);
                else
                    delta_u_omega = u(i, j)*modOmegaGrid(i, j) - u(i-1,
j)*modOmegaGrid(i-1, j);
                end
                if v(i, j) < 0
                    delta_v_omega = v(i, j+1)*modOmegaGrid(i, j+1) - v(i,
j)*modOmegaGrid(i, j);
                else
                    delta_v_omega = v(i, j)*modOmegaGrid(i, j) - v(i,
j-1)*modOmegaGrid(i, j-1);
                end
            end
        end
    end
end

```

```

        laplacian_omega = (modOmegaGrid(i+1, j) + modOmegaGrid(i-1, j) +
modOmegaGrid(i, j-1) + modOmegaGrid(i, j+1) - 4*modOmegaGrid(i, j))/(h^2);
        modOmegaGrid(i, j) = modOmegaGrid(i, j) +
t*(-1*(delta_u_omega)/h - (delta_v_omega)/h + nu*laplacian_omega);
    end
end
end
modOmegaGrid(n, :) = modOmegaGrid(n-1, :);
newGrid = modOmegaGrid;
end
%% iterate through temp grid
function newGrid = iterate_tempgrid(objGrid, oldTempGrid, u, v, h, t, alpha, n,
m)
    modTempGrid = oldTempGrid;
    for j = 2 : m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)
                if u(i, j) < 0
                    delta_u_temp = u(i, j)*modTempGrid(i+1, j) - u(i,
j)*modTempGrid(i, j);
                else
                    delta_u_temp = u(i, j)*modTempGrid(i, j) - u(i,
j)*modTempGrid(i-1, j);
                end
                if v(i, j) < 0
                    delta_v_temp = v(i, j)*modTempGrid(i, j+1) - v(i,
j)*modTempGrid(i, j);
                else
                    delta_v_temp = v(i, j)*modTempGrid(i, j) - v(i,
j)*modTempGrid(i, j-1);
                end
                laplacian_temp = (modTempGrid(i+1, j) + modTempGrid(i-1, j) +
modTempGrid(i, j-1) + modTempGrid(i, j+1) - 4*modTempGrid(i, j))/(h^2);
                modTempGrid(i, j) = modTempGrid(i, j) + t*(-1*(delta_u_temp)/h -
(delta_v_temp)/h + alpha*laplacian_temp);
            end
        end
    end
    modTempGrid(n, :) = modTempGrid(n-1, :);
    newGrid = modTempGrid;
end
%% iterate through psi grid using over-relaxed finite-difference (Gauss-Seidel)
and also error
function [newGrid, max_error] = iterate_psigrid(oldPsiGrid, objGrid, omegaGrid,
h, n, m, F)
    modPsiGrid = oldPsiGrid;
    errorGrid = zeros(n, m);
    for j = 2 : m - 1
        for i = 2 : n - 1

```



```

        if objGrid(i, j)
            continue;
        end
        % determine psi
        modPsiGrid(i, j) = modPsiGrid(i, j) + F*0.25*(modPsiGrid(i-1, j) +
modPsiGrid(i+1, j) + modPsiGrid(i, j-1) + modPsiGrid(i, j+1) +
4*h*h*omegaGrid(i, j) - 4*modPsiGrid(i, j));
        % determine error
        errorGrid(i, j) = abs(modPsiGrid(i, j) - oldPsiGrid(i, j)) /
oldPsiGrid(i, j);
    end
    modPsiGrid(n, j) = 2*modPsiGrid(n-1, j) - modPsiGrid(n-2, j);
end
newGrid = modPsiGrid;
max_error = max(errorGrid, [], 'all');
end

```

3) Plane with elliptical wings:

```

% MAE 423 Final Project
%
% By Dini Kathriarachchi & Allison Lesser
%
% set constants for problem
n = 500; % vertical length
m = 200; % horizontal length
threshold_error = 0.05;
F_overrelax = 1.5;
time_intervals = 10000;
numFrame = 0;
frame_store_rate = 10; % how many frames are stored
% constants for air
U_inf = 5; % m/s
T_inf = 300; % K
therm_diffusivity = 2.2 * 10^-5; % m^2/s
kin_viscosity = 1.5 * 10^-5;
Re = 200; % no units
delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf);
% create cell to store frames
psicell = cell(ceil(time_intervals/frame_store_rate), 1);
omegacell = cell(ceil(time_intervals/frame_store_rate), 1);
tempcell = cell(ceil(time_intervals/frame_store_rate), 1);
% create grids of desired geometry (n x m)
psiGrid = zeros(n, m);
omegaGrid = zeros(n, m);
uGrid = zeros(n, m);
vGrid = zeros(n, m);
tempGrid = T_inf*ones(n, m);

```

```

objGrid = false(n, m);
%
% ellipsoid wings
wlength = 50;
wwidth = 15;
coord_x = 100;
for i = 1 : n
    for j = 1 : m
        if ((j - ceil(m/2))^2/(wlength/2)^2 + (i - coord_x)^2/(wwidth/2)^2 <= 1)
            objGrid(i, j) = true;
        end
    end
end
% plane body
plength = 15;
pwidth = 75;
coord_x = 110;
for i = 1 : n
    for j = 1 : m
        if ((j - ceil(m/2))^2/(plength/2)^2 + (i - coord_x)^2/(pwidth/2)^2 <= 1)
            objGrid(i, j) = true;
        end
    end
end
% set boundary conditions
for j = 1 : m % left and right boundaries
    psiGrid(1, j) = (j - ceil(m/2)) * delta_h * U_inf;
    psiGrid(n, j) = (j - ceil(m/2)) * delta_h * U_inf;
end
% top and bottom boundaries
psiGrid(:, 1) = (1 - ceil(m/2)) * delta_h * U_inf;
psiGrid(:, m) = (m - ceil(m/2)) * delta_h * U_inf;
% temp boundaries
for j = 2 : m - 1
    for i = 2 : n - 1
        if objGrid(i, j)
            tempGrid(i, j) = 400;
        end
    end
end
%% model changes begin
for time = 1 : time_intervals
    % update u and v values for omega calculation
    for j = 2 : m - 1
        for i = 2 : n - 1
            uGrid(i, j) = (psiGrid(i, j+1) - psiGrid(i, j-1))/(2*delta_h);
            vGrid(i, j) = (psiGrid(i-1, j) - psiGrid(i+1, j))/(2*delta_h);
        end
    end
end

```

```

    % iterate through omega grid until errors are below threshold
    omegaGrid = iterate_omegagrid(psiGrid, objGrid, omegaGrid, uGrid, vGrid,
delta_h, delta_t, kin_viscosity, n, m);
    % iterate through omega grid until errors are below threshold
    tempGrid = iterate_tempgrid(objGrid, tempGrid, uGrid, vGrid, delta_h,
delta_t, therm_diffusivity, n, m);
    % iterate through psi grid until errors are below threshold
    max_error = Inf;
    while max_error > threshold_error
        [psiGrid, max_error] = iterate_psigrid(psiGrid, objGrid, omegaGrid,
delta_h, n, m, F_overrelax);
    end
    % store every 1 in few frames to avoid overloading :)
    if mod(time, frame_store_rate) == 1
        numFrame = numFrame + 1;
        psicell{numFrame} = psiGrid;
        omegacell{numFrame} = omegaGrid;
        tempcell{numFrame} = tempGrid;
        disp("Created frame " + string(numFrame) + " at time " +
string(datetime('now')));
    end
end
%
%% create video
videoFinal = VideoWriter('EllipsoidVideoMAE423.mp4', 'MPEG-4');
open(videoFinal);
% loop through frames and store them
for f = 1 : numFrame
    psi = psicell{f}';
    omega = flipud(omegacell{f}');
    temp = tempcell{f}';
    subplot(3, 1, 1)
    cla;
    imagesc(psi);
    hold on
    contour(psi, 40, 'k');
    hold off
    colormap(jet);
    axis equal tight;
    colorbar;
    title('Streamfunction');
    drawnow;
    subplot(3, 1, 2)
    cla;
    imagesc(omega)
    colormap(jet);
    axis equal tight;
    ax = gca; % Get current axes handle
    ax.CLim = [-0.05, 0.05]; % Set color limits

```

```

colorbar;
title('Vorticity');
drawnow;
subplot(3, 1, 3)
cla;
images(temp)
colormap(jet);
axis equal tight;
colorbar;
title('Temperature');
drawnow;
sgtitle("Time step No. " + string(f*frame_store_rate - 4));
frame = getframe(gcf); % get current frame
writeVideo(videoFinal, frame); % add frame to video
end
close(videoFinal);
%% iterate through omega grid
function newGrid = iterate_omegagrid(psiGrid, objGrid, oldOmegaGrid, u, v, h,
t, nu, n, m)
    modOmegaGrid = oldOmegaGrid;
    for j = 2 : m - 1
        for i = 2 : n - 1
            if objGrid(i, j)
                modOmegaGrid(i, j) = -2*(psiGrid(i+1, j) + psiGrid(i-1, j) +
psiGrid(i, j+1) + psiGrid(i, j-1) - 4*psiGrid(i, j))/(h^2);
            end
        end
    end
    for j = 2 : m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)
                if u(i, j) < 0
                    delta_u_omega = u(i+1, j)*modOmegaGrid(i+1, j) - u(i,
j)*modOmegaGrid(i, j);
                else
                    delta_u_omega = u(i, j)*modOmegaGrid(i, j) - u(i-1,
j)*modOmegaGrid(i-1, j);
                end
                if v(i, j) < 0
                    delta_v_omega = v(i, j+1)*modOmegaGrid(i, j+1) - v(i,
j)*modOmegaGrid(i, j);
                else
                    delta_v_omega = v(i, j)*modOmegaGrid(i, j) - v(i,
j-1)*modOmegaGrid(i, j-1);
                end
                laplacian_omega = (modOmegaGrid(i+1, j) + modOmegaGrid(i-1, j) +
modOmegaGrid(i, j-1) + modOmegaGrid(i, j+1) - 4*modOmegaGrid(i, j))/(h^2);
                modOmegaGrid(i, j) = modOmegaGrid(i, j) +
t*(-1*(delta_u_omega)/h - (delta_v_omega)/h + nu*laplacian_omega);
            end
        end
    end
end

```

```

        end
    end
    end
    modOmegaGrid(n, :) = modOmegaGrid(n-1, :);
    newGrid = modOmegaGrid;
end
%% iterate through temp grid
function newGrid = iterate_tempgrid(objGrid, oldTempGrid, u, v, h, t, alpha, n,
m)
    modTempGrid = oldTempGrid;
    for j = 2 : m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)
                if u(i, j) < 0
                    delta_u_temp = u(i, j)*modTempGrid(i+1, j) - u(i,
j)*modTempGrid(i, j);
                else
                    delta_u_temp = u(i, j)*modTempGrid(i, j) - u(i,
j)*modTempGrid(i-1, j);
                end
                if v(i, j) < 0
                    delta_v_temp = v(i, j)*modTempGrid(i, j+1) - v(i,
j)*modTempGrid(i, j);
                else
                    delta_v_temp = v(i, j)*modTempGrid(i, j) - v(i,
j)*modTempGrid(i, j-1);
                end
                laplacian_temp = (modTempGrid(i+1, j) + modTempGrid(i-1, j) +
modTempGrid(i, j-1) + modTempGrid(i, j+1) - 4*modTempGrid(i, j))/(h^2);
                modTempGrid(i, j) = modTempGrid(i, j) + t*(-1*(delta_u_temp)/h -
(delta_v_temp)/h + alpha*laplacian_temp);
            end
        end
    end
    modTempGrid(n, :) = modTempGrid(n-1, :);
    newGrid = modTempGrid;
end
%% iterate through psi grid using over-relaxed finite-difference (Gauss-Seidel)
and also error
function [newGrid, max_error] = iterate_psigrid(oldPsiGrid, objGrid, omegaGrid,
h, n, m, F)
    modPsiGrid = oldPsiGrid;
    errorGrid = zeros(n, m);
    for j = 2 : m - 1
        for i = 2 : n - 1
            if objGrid(i, j)
                continue;
            end
            % determine psi

```

```

        modPsiGrid(i, j) = modPsiGrid(i, j) + F*0.25*(modPsiGrid(i-1, j) +
modPsiGrid(i+1, j) + modPsiGrid(i, j-1) + modPsiGrid(i, j+1) +
4*h*h*omegaGrid(i, j) - 4*modPsiGrid(i, j));
        % determine error
        errorGrid(i, j) = abs(modPsiGrid(i, j) - oldPsiGrid(i, j)) /
oldPsiGrid(i, j);
    end
    modPsiGrid(n, j) = 2*modPsiGrid(n-1, j) - modPsiGrid(n-2, j);
end
newGrid = modPsiGrid;
max_error = max(errorGrid, [], 'all');
end

```

4) Plane with tapered wings:

```

% MAE 423 Final Project
%
% By Dini Kathriarachchi & Allison Lesser
%
% set constants for problem
n = 500; % vertical length
m = 200; % horizontal length
threshold_error = 0.05;
F_overrelax = 1.5;
time_intervals = 10000;
numFrame = 0;
frame_store_rate = 10; % how many frames are stored
% constants for air
U_inf = 5; % m/s
T_inf = 300; % K
therm_diffusivity = 2.2 * 10^-5; % m^2/s
kin_viscosity = 1.5 * 10^-5;
Re = 200; % no units
delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf);
% create cell to store frames
psicell = cell(ceil(time_intervals/frame_store_rate), 1);
omegacell = cell(ceil(time_intervals/frame_store_rate), 1);
tempcell = cell(ceil(time_intervals/frame_store_rate), 1);
% create grids of desired geometry (n x m)
psiGrid = zeros(n, m);
omegaGrid = zeros(n, m);
uGrid = zeros(n, m);
vGrid = zeros(n, m);
tempGrid = T_inf*ones(n, m);
objGrid = false(n, m);
%
% tapered wings
base_height = 10;

```

```

wlength = 50;
coord_x = 95;
for i = coord_x - int32(2/3*base_height) : coord_x + int32(1/3*base_height)
    y = i - coord_x + int32(2/3*base_height);
    for j = 1 : m
        if (abs(j - ceil(m/2)) <= y*0.5*wlength/base_height)
            objGrid(i, j) = true;
        end
    end
end
wlength = 50;
wwidth = 10;
coord_x = 100;
for i = coord_x - ceil(wwidth/3) : coord_x + floor(2*wwidth/3)
    for j = ceil(m/2) - ceil(wlength/2) : ceil(m/2) + floor(wlength/2)
        objGrid(i, j) = true;
    end
end
% plane body
plength = 15;
pwidth = 75;
coord_x = 110;
for i = 1 : n
    for j = 1 : m
        if ((j - ceil(m/2))^2/(plength/2)^2 + (i - coord_x)^2/(pwidth/2)^2 <= 1)
            objGrid(i, j) = true;
        end
    end
end
% set boundary conditions
for j = 1 : m % left and right boundaries
    psiGrid(1, j) = (j - ceil(m/2)) * delta_h * U_inf;
    psiGrid(n, j) = (j - ceil(m/2)) * delta_h * U_inf;
end
% top and bottom boundaries
psiGrid(:, 1) = (1 - ceil(m/2)) * delta_h * U_inf;
psiGrid(:, m) = (m - ceil(m/2)) * delta_h * U_inf;
% temp boundaries
for j = 2 : m - 1
    for i = 2 : n - 1
        if objGrid(i, j)
            tempGrid(i, j) = 400;
        end
    end
end
%% model changes begin
for time = 1 : time_intervals
    % update u and v values for omega calculation
    for j = 2 : m - 1

```

```

        for i = 2 : n - 1
            uGrid(i, j) = (psiGrid(i, j+1) - psiGrid(i, j-1))/(2*delta_h);
            vGrid(i, j) = (psiGrid(i-1, j) - psiGrid(i+1, j))/(2*delta_h);
        end
    end
    % iterate through omega grid until errors are below threshold
    omegaGrid = iterate_omegagrid(psiGrid, objGrid, omegaGrid, uGrid, vGrid,
    delta_h, delta_t, kin_viscosity, n, m);
    % iterate through omega grid until errors are below threshold
    tempGrid = iterate_tempgrid(objGrid, tempGrid, uGrid, vGrid, delta_h,
    delta_t, therm_diffusivity, n, m);
    % iterate through psi grid until errors are below threshold
    max_error = Inf;
    while max_error > threshold_error
        [psiGrid, max_error] = iterate_psigrid(psiGrid, objGrid, omegaGrid,
    delta_h, n, m, F_overrelax);
    end
    % store every 1 in few frames to avoid overloading :)
    if mod(time, frame_store_rate) == 1
        numFrame = numFrame + 1;
        psicell{numFrame} = psiGrid;
        omegacell{numFrame} = omegaGrid;
        tempcell{numFrame} = tempGrid;
        disp("Created frame " + string(numFrame) + " at time " +
    string(datetime('now')));
    end
end
%
%% create video
videoFinal = VideoWriter('TaperedVideoMAE423.mp4', 'MPEG-4');
open(videoFinal);
% loop through frames and store them
for f = 1 : numFrame
    psi = psicell{f}';
    omega = flipud(omegacell{f}');
    temp = tempcell{f}';
    subplot(3, 1, 1)
    cla;
    imagesc(psi);
    hold on
    contour(psi, 40, 'k');
    hold off
    colormap(jet);
    axis equal tight;
    colorbar;
    title('Streamfunction');
    drawnow;
    subplot(3, 1, 2)
    cla;

```



```

imagesc(omega)
colormap(jet);
axis equal tight;
ax = gca; % Get current axes handle
ax.CLim = [-0.05, 0.05]; % Set color limits
colorbar;
title('Vorticity');
drawnow;
subplot(3, 1, 3)
cla;
imagesc(temp)
colormap(jet);
axis equal tight;
colorbar;
title('Temperature');
drawnow;
sgtitle("Time step No. " + string(f*frame_store_rate - 4));
frame = getframe(gcf); % get current frame
writeVideo(videoFinal, frame); % add frame to video
end
close(videoFinal);
%% iterate through omega grid
function newGrid = iterate_omegagrid(psiGrid, objGrid, oldOmegaGrid, u, v, h,
t, nu, n, m)
    modOmegaGrid = oldOmegaGrid;
    for j = 2 : m - 1
        for i = 2 : n - 1
            if objGrid(i, j)
                modOmegaGrid(i, j) = -2*(psiGrid(i+1, j) + psiGrid(i-1, j) +
psiGrid(i, j+1) + psiGrid(i, j-1) - 4*psiGrid(i, j))/(h^2);
            end
        end
    end
    for j = 2 : m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)
                if u(i, j) < 0
                    delta_u_omega = u(i+1, j)*modOmegaGrid(i+1, j) - u(i,
j)*modOmegaGrid(i, j);
                else
                    delta_u_omega = u(i, j)*modOmegaGrid(i, j) - u(i-1,
j)*modOmegaGrid(i-1, j);
                end
                if v(i, j) < 0
                    delta_v_omega = v(i, j+1)*modOmegaGrid(i, j+1) - v(i,
j)*modOmegaGrid(i, j);
                else
                    delta_v_omega = v(i, j)*modOmegaGrid(i, j) - v(i,
j-1)*modOmegaGrid(i, j-1);

```

```

        end
        laplacian_omega = (modOmegaGrid(i+1, j) + modOmegaGrid(i-1, j) +
modOmegaGrid(i, j-1) + modOmegaGrid(i, j+1) - 4*modOmegaGrid(i, j))/(h^2);
        modOmegaGrid(i, j) = modOmegaGrid(i, j) +
t*(-1*(delta_u_omega)/h - (delta_v_omega)/h + nu*laplacian_omega);
    end
end
end
modOmegaGrid(n, :) = modOmegaGrid(n-1, :);
newGrid = modOmegaGrid;
end
%% iterate through temp grid
function newGrid = iterate_tempgrid(objGrid, oldTempGrid, u, v, h, t, alpha, n,
m)
    modTempGrid = oldTempGrid;
    for j = 2 : m - 1
        for i = 2 : n - 1
            if ~objGrid(i, j)
                if u(i, j) < 0
                    delta_u_temp = u(i, j)*modTempGrid(i+1, j) - u(i,
j)*modTempGrid(i, j);
                else
                    delta_u_temp = u(i, j)*modTempGrid(i, j) - u(i,
j)*modTempGrid(i-1, j);
                end
                if v(i, j) < 0
                    delta_v_temp = v(i, j)*modTempGrid(i, j+1) - v(i,
j)*modTempGrid(i, j);
                else
                    delta_v_temp = v(i, j)*modTempGrid(i, j) - v(i,
j)*modTempGrid(i, j-1);
                end
                laplacian_temp = (modTempGrid(i+1, j) + modTempGrid(i-1, j) +
modTempGrid(i, j-1) + modTempGrid(i, j+1) - 4*modTempGrid(i, j))/(h^2);
                modTempGrid(i, j) = modTempGrid(i, j) + t*(-1*(delta_u_temp)/h -
(delta_v_temp)/h + alpha*laplacian_temp);
            end
        end
    end
    modTempGrid(n, :) = modTempGrid(n-1, :);
    newGrid = modTempGrid;
end
%% iterate through psi grid using over-relaxed finite-difference (Gauss-Seidel)
and also error
function [newGrid, max_error] = iterate_psigrid(oldPsiGrid, objGrid, omegaGrid,
h, n, m, F)
    modPsiGrid = oldPsiGrid;
    errorGrid = zeros(n, m);
    for j = 2 : m - 1

```

```

    for i = 2 : n - 1
        if objGrid(i, j)
            continue;
        end
        % determine psi
        modPsiGrid(i, j) = modPsiGrid(i, j) + F*0.25*(modPsiGrid(i-1, j) +
modPsiGrid(i+1, j) + modPsiGrid(i, j-1) + modPsiGrid(i, j+1) +
4*h*h*omegaGrid(i, j) - 4*modPsiGrid(i, j));
        % determine error
        errorGrid(i, j) = abs(modPsiGrid(i, j) - oldPsiGrid(i, j)) /
oldPsiGrid(i, j);
    end
    modPsiGrid(n, j) = 2*modPsiGrid(n-1, j) - modPsiGrid(n-2, j);
end
newGrid = modPsiGrid;
max_error = max(errorGrid, [], 'all');
end

```

5) Nusselt number calculations:

```

n = 500; % vertical length
m = 200; % horizontal length
numFrame = 1000;
c_air = 1005; % J/kgK
density_air = 1.17; % kg/m3
k_air = 0.024; % W/mK
boundary_right = 500;
boundary_left = 1;
Q_over_A = zeros(numFrame, m);
% constants for air
U_inf = 5; % m/s
kin_viscosity = 1.5 * 10^-5;
Re = 200; % no units
delta_h = Re * kin_viscosity / (50*U_inf); %
delta_t = delta_h/(10*U_inf);
L = delta_h * 50;
for f = 2 : numFrame % go through all frames
    psi = taper_psiCell{f}; % get stored psi values for each frame
    temp = taper_tempCell{f}; % get stored temp values for each frame
    for j = 2 : m-1 % bottom to top
        u_right = (psi(boundary_right, j+1) - psi(boundary_right,
j-1))/(2*delta_h); % calculate velocity at right boundary at each point
        u_left = (psi(boundary_left, j+1) - psi(boundary_left,
j-1))/(2*delta_h); % calculate velocity at left boundary (should be = u_inf)
        T_right = temp(boundary_right, j);
        T_left = temp(boundary_left, j);
        Q_over_A(f, j) = density_air * c_air * (T_right*u_right -
T_left*u_left);
        % kg/m3 * J/kgK * K*m/s = J/(s*m2) = W / m2 = Q/A
    end
end

```

```
end
avg_heat_transfer = mean(Q_over_A, 'all'); % averaging across all frames and all
vertical grid cells
h_convect = avg_heat_transfer / (400 - 300);
Nusselt = h_convect * L / k_air;
```